

eBPF Explorer: A Real-Time Linux System Call and Process Activity Monitor

Submitted by:

A.K.M. Zahid Hossain - 2201011018

Inkiyed Bin Taher - 2201011014

Dhurjoy Dey - 2201011004

Department of Computer Science and Engineering

Operating Systems Lab Project

Submitted To:

Md. Mynoddin

Assistant Professor

Department of CSE

Rangamati Science & Technology University

June 19, 2026

Abstract

This report presents the design and implementation of **eBPF Explorer**, a lightweight system monitoring tool built using the extended Berkeley Packet Filter (eBPF) technology and the BPF Compiler Collection (BCC) framework on Kali Linux. The tool attaches kernel tracepoints to observe process creation (`fork`), program execution (`exec`), file access (`openat`), and CPU scheduling (`sched_switch`) events in real time. A combined dashboard integrates `fork`, `exec`, and `open` monitoring into a single interface. The project demonstrates practical kernel-level observability without modifying kernel source code or requiring kernel module compilation, illustrating the power of eBPF for system tracing and security monitoring.

Each monitoring component was developed as a standalone Python script and tested independently before being combined into the final dashboard. The scripts were written entirely using the `nano` terminal editor on Kali Linux, following the lab requirement of avoiding graphical IDEs. The project was version-controlled using Git, with each development milestone recorded as a separate commit and published to a public GitHub repository. Results show that the tool successfully captures real-time kernel events with minimal system overhead, making it suitable for both educational purposes and lightweight security auditing on Linux systems.

Contents

1	Introduction	3
2	Background	3
2.1	eBPF (Extended Berkeley Packet Filter)	3
2.2	BCC Framework	3
2.3	Tracepoints	3
2.4	trace_pipe and trace_print()	4
3	System Design	4
4	Implementation	4
4.1	Fork Monitor (<code>fork_monitor.py</code>)	5
4.2	Exec Monitor (<code>exec_monitor.py</code>)	5
4.3	Open Syscall Monitor (<code>open_monitor.py</code>)	6
4.4	Scheduler Monitor (<code>scheduler_monitor.py</code>)	6
4.5	Combined Dashboard (<code>dashboard.py</code>)	7
5	Demonstration and Results	8
6	Version Control and Project Repository	10
7	Challenges Faced	11
8	Conclusion and Future Work	11

1 Introduction

Modern operating systems execute thousands of events every second — processes are created and destroyed, programs are executed, files are opened, and the CPU scheduler constantly switches between tasks. Traditionally, observing such low-level kernel activity required tools such as `strace`, `ltrace`, or custom kernel modules, all of which introduce significant overhead or require deep kernel programming expertise.

eBPF (extended Berkeley Packet Filter) is a modern Linux kernel technology that allows small, sandboxed programs to run safely inside the kernel in response to events, without changing kernel source code or loading traditional kernel modules. The **BCC (BPF Compiler Collection)** framework provides Python bindings that compile C-like eBPF programs at runtime and attach them to kernel **tracepoints**.

This project, **eBPF Explorer**, implements a set of Python scripts that use BCC to monitor:

- Process `fork` events (parent and child PIDs)
- Process `exec` events (program execution)
- File `open` (`openat`) syscalls
- CPU scheduler context switches

These individual monitors are then combined into a unified **dashboard** that displays `fork`, `exec`, and `open` events together in real time.

2 Background

2.1 eBPF (Extended Berkeley Packet Filter)

eBPF allows user-defined programs to be loaded into the Linux kernel and executed in response to specific events (such as system calls, function entry/exit, or network packets). Each eBPF program is verified by an in-kernel verifier to guarantee safety (no infinite loops, no invalid memory access) before being JIT-compiled and executed.

2.2 BCC Framework

BCC (BPF Compiler Collection) is a toolkit that simplifies eBPF program development. It allows developers to write the kernel-side logic in C and the user-space control logic in Python. The `bcc.BPF` class compiles the embedded C code using Clang/LLVM at runtime and attaches it to the requested kernel hooks.

2.3 Tracepoints

Tracepoints are static instrumentation points placed at meaningful locations in the kernel source code (e.g., `sched:sched_process_fork`, `syscalls:sys_enter_openat`). eBPF programs can attach to these tracepoints using the `TRACEPOINT_PROBE` macro provided by BCC, gaining access to event-specific arguments through the `args` structure.

2.4 `trace_pipe` and `trace_print()`

Output from `bpf_trace_printk()` calls inside the eBPF program is written to the kernel's `/sys/kernel/debug/tracing/trace_pipe`. BCC's `b.trace_print()` function continuously reads and prints this pipe to the terminal, providing a simple real-time output mechanism.

3 System Design

The project follows an incremental, modular design. Each monitoring capability is implemented as an independent script, and the final dashboard combines the most important probes into a single program. The overall architecture can be summarized as follows:

1. **User space (Python + BCC):** Defines the eBPF C program as a string, compiles and loads it via `BPF(text=program)`.
2. **Kernel space (eBPF program):** Attached to tracepoints; executes `bpf_trace_printk()` when the event occurs.
3. **`trace_pipe`:** Kernel buffer that receives the `printk` output.
4. **User space (Python):** `b.trace_print()` reads `trace_pipe` and prints formatted output to the terminal.

4 Implementation

All scripts were developed incrementally on Kali Linux using `nano` as the editor and `sudo python3` to execute (eBPF programs require root privileges to load into the kernel).

The development process followed a bottom-up approach: simpler monitors (`fork`, `exec`) were implemented and verified first, before progressing to more complex ones (`open syscall` with filename extraction). This strategy allowed each component to be tested in isolation, making debugging straightforward. Every script follows the same structural pattern: an embedded C string containing the eBPF tracepoint probe, a `BPF(text=program)` call to compile and load it, and a `trace_print()` loop to display the output. This consistent structure makes the codebase easy to extend with additional tracepoints in the future.

4.1 Fork Monitor (fork_monitor.py)

This script attaches to the `sched_process_fork` tracepoint, which fires whenever a process creates a child process via `fork()/clone()`. It reports the parent and child process IDs.

Listing 1: fork_monitor.py

```
1 from bcc import BPF
2
3 program = """
4 #include <uapi/linux/ptrace.h>
5
6 TRACEPOINT_PROBE(sched, sched_process_fork)
7 {
8     bpf_trace_printk("Fork: parent=%d child=%d\\n", args->parent_pid,
9         args->child_pid);
10    return 0;
11 }
12 """
13 b = BPF(text=program)
14
15 print("Listening for fork events with PID info...")
16
17 b.trace_print()
```

4.2 Exec Monitor (exec_monitor.py)

This script attaches to the `sched_process_exec` tracepoint, which fires whenever a process replaces its memory image via `execve()` (i.e., when a new program starts running). The current process ID is obtained using `bpf_get_current_pid_tgid()`.

Listing 2: exec_monitor.py

```
1 from bcc import BPF
2
3 program = """
4 #include <uapi/linux/ptrace.h>
5
6 TRACEPOINT_PROBE(sched, sched_process_exec)
7 {
8     bpf_trace_printk("Exec: pid=%d\\n", bpf_get_current_pid_tgid() >>
9         32);
10    return 0;
11 }
12 """
13 b = BPF(text=program)
14
15 print("Listening for exec events...")
16
17 b.trace_print()
```

4.3 Open Syscall Monitor (open_monitor.py)

This script attaches to `syscalls:sys_enter_openat`, which fires whenever a process opens a file using the `openat()` system call. The filename argument, which resides in user-space memory, is safely copied into kernel space using `bpf_probe_read_user_str()`.

Listing 3: open_monitor.py

```
1 from bcc import BPF
2
3 program = """
4 #include <uapi/linux/ptrace.h>
5
6 TRACEPOINT_PROBE(syscalls, sys_enter_openat)
7 {
8     char fname[64];
9     bpf_probe_read_user_str(&fname, sizeof(fname), (void *)args->
10         filename);
11     bpf_trace_printk("Open: pid=%d file=%s\\n",
12         bpf_get_current_pid_tgid() >> 32, fname);
13     return 0;
14 }
15 """
16
17 b = BPF(text=program)
18
19 print("Listening for file open events...")
20
21 b.trace_print()
```

4.4 Scheduler Monitor (scheduler_monitor.py)

This script attaches to `sched:sched_switch`, which fires on every CPU context switch, reporting the name of the process being switched out (`prev_comm`) and the process being switched in (`next_comm`).

Listing 4: scheduler_monitor.py

```
1 from bcc import BPF
2
3 program = """
4 TRACEPOINT_PROBE(sched, sched_switch)
5 {
6     bpf_trace_printk("Sched switch: prev=%s next=%s\\n", args->
7         prev_comm, args->next_comm);
8     return 0;
9 }
10 """
11
12 b = BPF(text=program)
13
14 print("Listening for scheduler switch events...")
15
16 b.trace_print()
```

4.5 Combined Dashboard (dashboard.py)

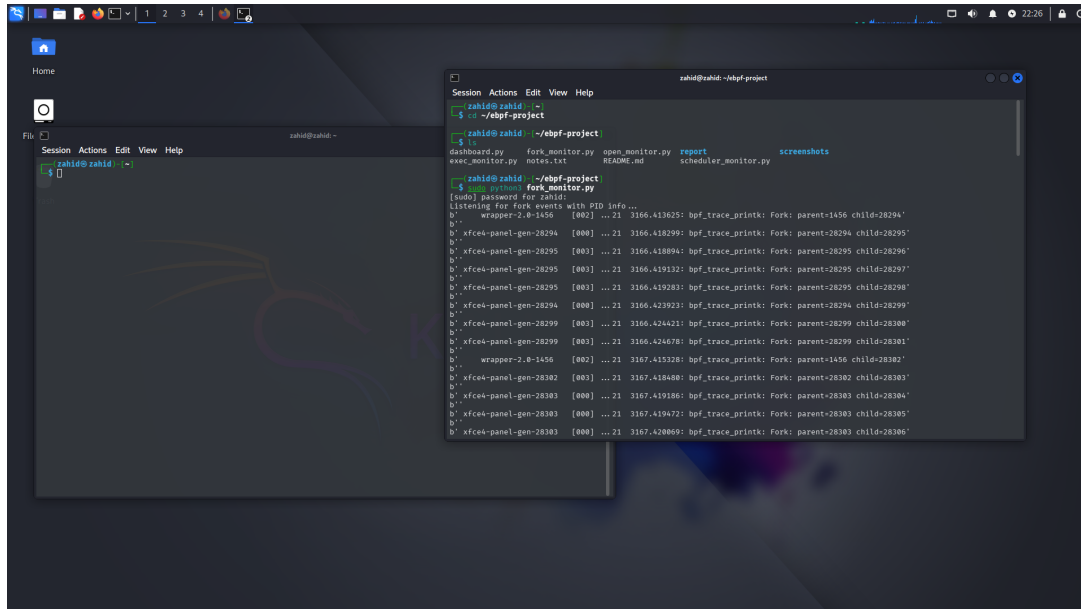
The dashboard combines the fork, exec, and open probes into a single eBPF program, providing a unified, real-time view of process and file activity on the system.

Listing 5: dashboard.py

```
1 from bcc import BPF
2
3 program = """
4 #include <uapi/linux/ptrace.h>
5
6 TRACEPOINT_PROBE(sched, sched_process_fork)
7 {
8     bpf_trace_printk("FORK parent=%d child=%d\\n", args->parent_pid,
9     args->child_pid);
10    return 0;
11 }
12
13 TRACEPOINT_PROBE(sched, sched_process_exec)
14 {
15     bpf_trace_printk("EXEC pid=%d\\n", bpf_get_current_pid_tgid() >>
16     32);
17     return 0;
18 }
19
20 TRACEPOINT_PROBE(syscalls, sys_enter_openat)
21 {
22     char fname[64];
23     bpf_probe_read_user_str(&fname, sizeof(fname), (void *)args->
24     filename);
25     bpf_trace_printk("OPEN pid=%d file=%s\\n", bpf_get_current_pid_tgid
26     () >> 32, fname);
27     return 0;
28 }
29 """
30
31 b = BPF(text=program)
32
33 print("=== eBPF Explorer Dashboard ===")
34 print("Monitoring: Fork | Exec | Open events")
35 print("Press Ctrl+C to stop\\n")
36
37 b.trace_print()
```

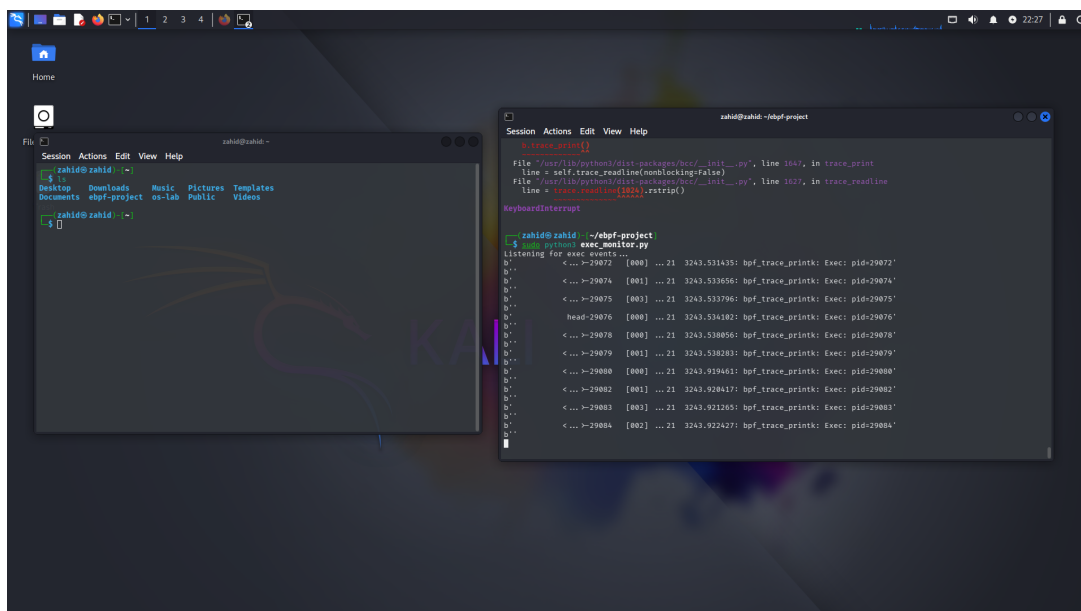
5 Demonstration and Results

Each script was executed using `sudo python3 <script>.py` while a second terminal generated system activity (e.g., `ls`, `cat`, `sleep 5 &`). The figures below show representative output captured during testing.



```
zahid@zahid: ~  
$ python3 fork_monitor.py  
[sudo] password for zahid:  
Listening for fork events with PID info...  
wrapper-2.0-1456 [002] ... 21 3166.413625: bpf_trace_printk: Fork: parent=1456 child=28294'  
b''  
b'' xfce4-panel-gen-28294 [000] ... 21 3166.418299: bpf_trace_printk: Fork: parent=28294 child=28295'  
b''  
b'' xfce4-panel-gen-28295 [003] ... 21 3166.418894: bpf_trace_printk: Fork: parent=28295 child=28296'  
b''  
b'' xfce4-panel-gen-28295 [003] ... 21 3166.419132: bpf_trace_printk: Fork: parent=28295 child=28297'  
b''  
b'' xfce4-panel-gen-28295 [003] ... 21 3166.419283: bpf_trace_printk: Fork: parent=28295 child=28298'  
b''  
b'' xfce4-panel-gen-28294 [000] ... 21 3166.423923: bpf_trace_printk: Fork: parent=28294 child=28299'  
b''  
b'' xfce4-panel-gen-28299 [003] ... 21 3166.424421: bpf_trace_printk: Fork: parent=28299 child=28300'  
b''  
b'' xfce4-panel-gen-28299 [003] ... 21 3166.424678: bpf_trace_printk: Fork: parent=28299 child=28301'  
b''  
b'' wrapper-2.0-1456 [002] ... 21 3167.415328: bpf_trace_printk: Fork: parent=1456 child=28302'  
b''  
b'' xfce4-panel-gen-28302 [003] ... 21 3167.418480: bpf_trace_printk: Fork: parent=28302 child=28303'  
b''  
b'' xfce4-panel-gen-28303 [000] ... 21 3167.419186: bpf_trace_printk: Fork: parent=28303 child=28304'  
b''  
b'' xfce4-panel-gen-28303 [000] ... 21 3167.419472: bpf_trace_printk: Fork: parent=28303 child=28305'  
b''  
b'' xfce4-panel-gen-28303 [000] ... 21 3167.420869: bpf_trace_printk: Fork: parent=28303 child=28306'
```

Figure 1: Fork Monitor output showing parent and child PIDs



```
zahid@zahid: ~  
$ python3 exec_monitor.py  
Listening for exec events...  
< ... > 29072 [000] ... 21 3243.531435: bpf_trace_printk: Exec: pid=29072'  
b''  
b'' < ... > 29074 [001] ... 21 3243.533656: bpf_trace_printk: Exec: pid=29074'  
b''  
b'' < ... > 29075 [003] ... 21 3243.533796: bpf_trace_printk: Exec: pid=29075'  
b''  
b'' head-29076 [000] ... 21 3243.534102: bpf_trace_printk: Exec: pid=29076'  
b''  
b'' < ... > 29078 [000] ... 21 3243.538056: bpf_trace_printk: Exec: pid=29078'  
b''  
b'' < ... > 29079 [001] ... 21 3243.538283: bpf_trace_printk: Exec: pid=29079'  
b''  
b'' < ... > 29080 [000] ... 21 3243.919451: bpf_trace_printk: Exec: pid=29080'  
b''  
b'' < ... > 29082 [001] ... 21 3243.920417: bpf_trace_printk: Exec: pid=29082'  
b''  
b'' < ... > 29083 [003] ... 21 3243.921205: bpf_trace_printk: Exec: pid=29083'  
b''  
b'' < ... > 29084 [002] ... 21 3243.922427: bpf_trace_printk: Exec: pid=29084'
```

Figure 2: Exec Monitor output showing process IDs of executed programs

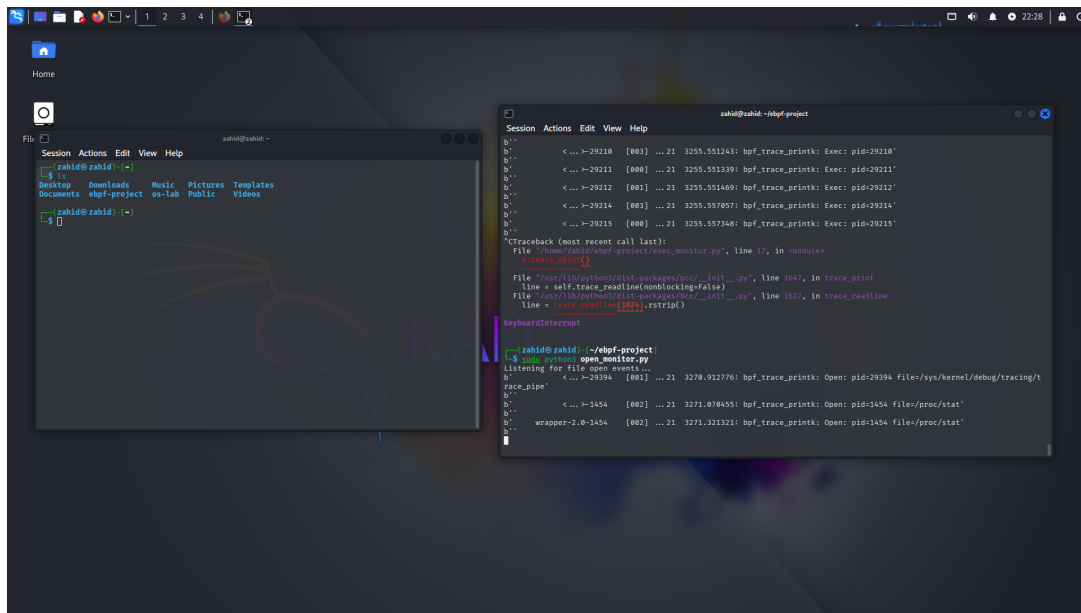


Figure 3: Open Monitor output showing files opened by various processes

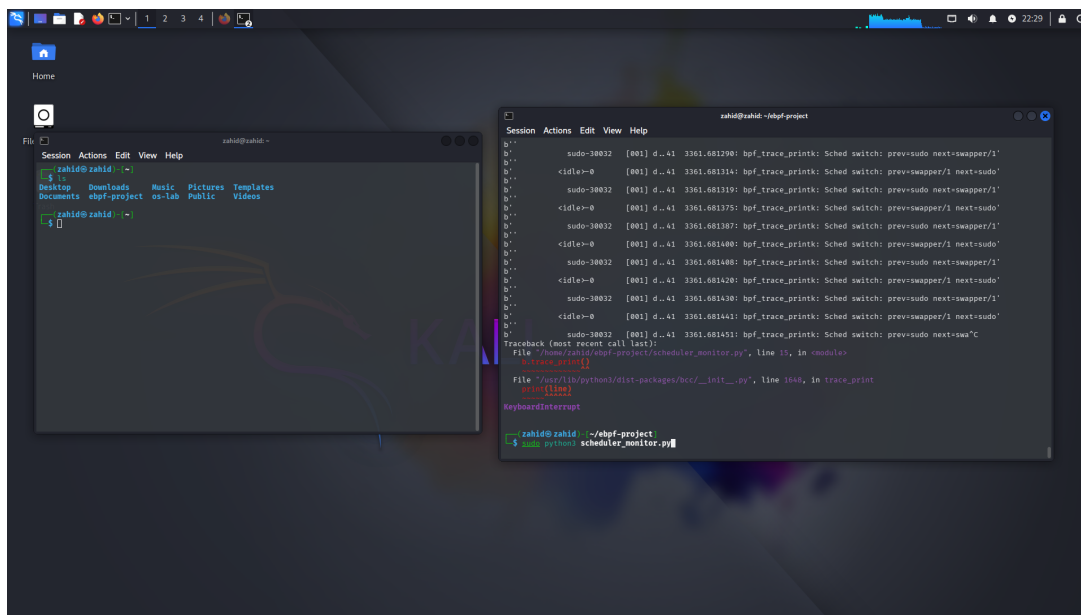


Figure 4: Scheduler Monitor output showing CPU context switches

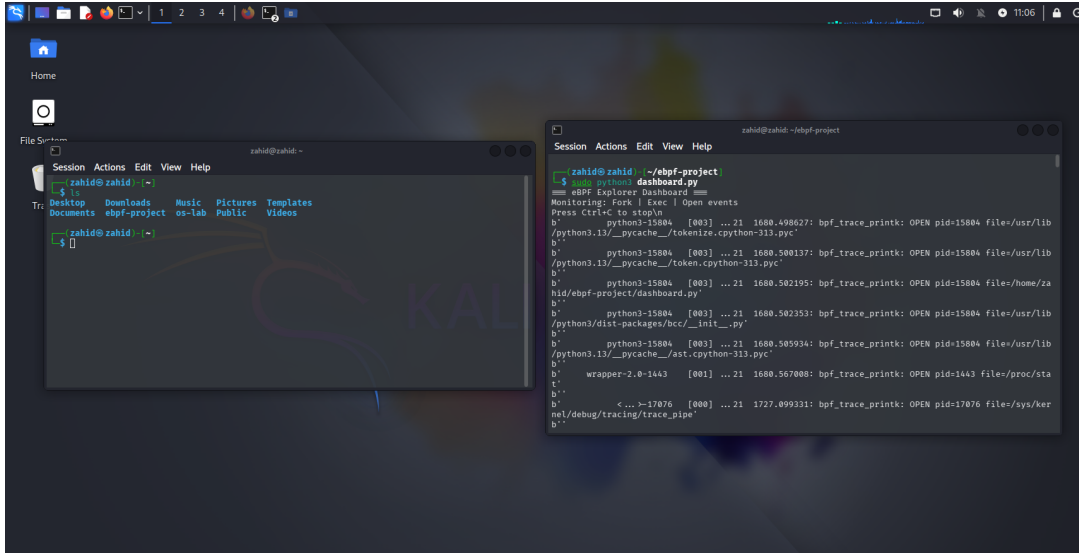


Figure 5: Combined Dashboard showing FORK, EXEC, and OPEN events together

6 Version Control and Project Repository

The project was developed incrementally, with each milestone recorded as a separate Git commit. The repository is publicly hosted on GitHub at:

GitHub Repository:

<https://github.com/Zaahid-7/ebpf-explorer>

The commit history reflects the step-by-step development process, progressing from the initial project structure through individual monitors to the final combined dashboard.

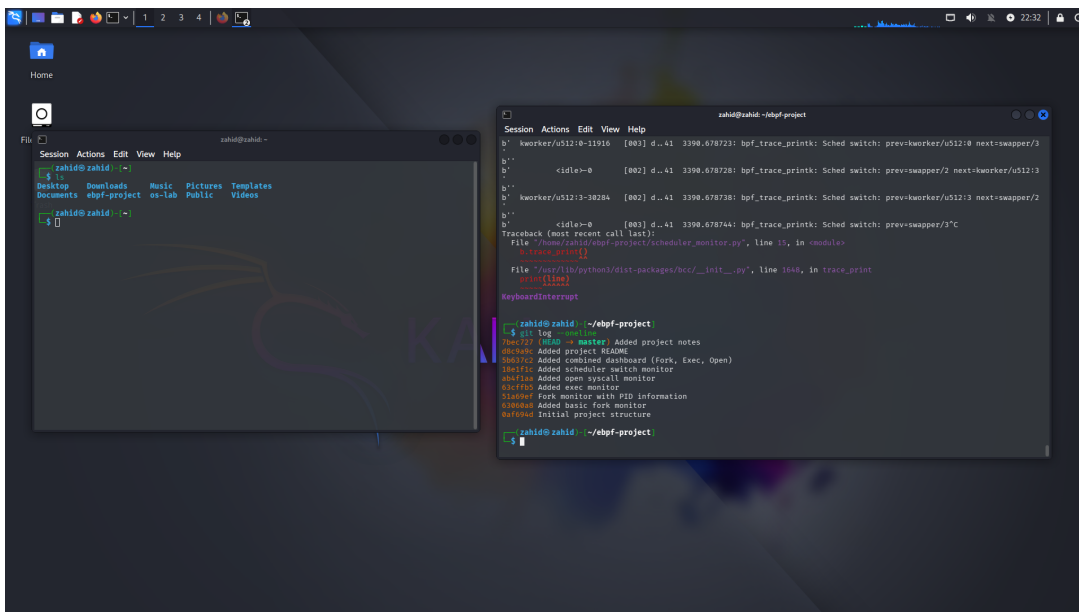


Figure 6: Git commit history showing incremental development

7 Challenges Faced

- **Permissions:** Loading eBPF programs into the kernel requires root privileges, so all scripts must be run using `sudo`.
- **Output interleaving:** Using `b.trace_print()` reads directly from `trace_pipe`, which can interleave output from unrelated kernel activity (e.g., desktop panel processes), requiring careful filtering when interpreting results.
- **Safe memory access:** Reading user-space strings (e.g., file paths) from within an eBPF program required using `bpf_probe_read_user_str()` rather than direct pointer dereference, due to the eBPF verifier's memory-safety restrictions.

8 Conclusion and Future Work

This project successfully demonstrates the use of eBPF and the BCC framework to build a lightweight, real-time system monitoring tool on Linux. Without modifying the kernel or writing traditional kernel modules, the tool can observe process creation, program execution, file access, and CPU scheduling activity directly from kernel tracepoints.

Future improvements could include:

- Filtering events by process name or PID
- Logging events to a file or database for offline analysis
- Adding a graphical (curses-based or web-based) dashboard
- Extending coverage to network-related syscalls (e.g., `connect`, `accept`)

References

- [1] IO Visor Project, *BCC – Tools for BPF-based Linux IO analysis, networking, monitoring, and more*, <https://github.com/iovisor/bcc>
- [2] Linux Kernel Documentation, *BPF Documentation*, <https://www.kernel.org/doc/html/latest/bpf/index.html>
- [3] Linux Kernel Documentation, *Tracepoints*, <https://www.kernel.org/doc/html/latest/trace/tracepoints.html>